PONTIFICIA UNIVERSIDAD CATÓLICA ARGENTINA

Facultad de Química e Ingeniería del Rosario

Licenciatura en Ciencias de Datos

Simulación y Procesos Estocásticos

Trabajo Práctico Grupal N° 1

Unidad Temática I: Probabilidad, Estadística
y Generación de Números Aleatorios

Integrantes del grupo:

Ancarani, Juan Diego

Chocobares, Juan Cruz

Formenti, Agustín

Mendes, Lorenzo

Morenico, Andrés

Rosario, 2026

Índice

1. Introducción **2**

2. Distribuciones Estadísticas **3**
2.1. Distribución Uniforme (continua) 3
2.2. Distribución Triangular (continua) 4
2.3. Distribución Exponencial (continua) 4
2.4. Distribución Bernoulli (discreta) 5
2.5. Distribución Binomial (discreta) 5
2.6. Distribución Poisson (discreta) 6
2.7. Distribución Normal (continua) 6

3. Métodos de Generación de Números Aleatorios **7**
3.1. Método de los Cuadrados Medios (Mid-Square) 7
3.2. Método Congruencial Lineal (LCG) 7

4. Variantes del Método Congruencial **9**
4.1. Congruencial Multiplicativo ($c = 0$) 9
4.2. Congruencial Aditivo ($a = 1$) 9
4.3. Congruencial Mixto (caso general) 9
4.4. Necesidad de los Tests de Verificación 10

5. Pruebas de Aleatoriedad **11**
5.1. Prueba de Uniformidad (Chi-cuadrado) 11
5.2. Prueba de Kolmogorov-Smirnov (KS) 11
5.3. Prueba de Independencia — Autocorrelación 11
5.4. Prueba de la Corrida (Runs Test) 12
5.5. Prueba de Repetitividad 12
5.6. Prueba de Uniformidad de los Dígitos 13
5.7. Prueba de Distancia 13

6. Generación de Variables Aleatorias **14**
6.1. Método de la Transformada Inversa 14
6.2. Distribución Exponencial 14
6.3. Distribución Uniforme (a, b) 14
6.4. Distribución Triangular 14
6.5. Distribuciones Discretas 15

7. Conclusiones **16**

Referencias **16**

1. Introducción

El presente trabajo práctico aborda los fundamentos teóricos y computacionales de la simulación de sistemas mediante la generación y análisis de variables aleatorias. La simulación es una herramienta fundamental en la Ciencia de Datos, ya que permite modelar el comportamiento de sistemas complejos cuya solución analítica es difícil o imposible de obtener.

Para simular el comportamiento de sistemas reales se recurre a variables aleatorias que responden a distintas distribuciones de probabilidad. Cada distribución captura un tipo diferente de fenómeno: algunas modelan eventos discretos con dos resultados posibles (Bernoulli), otras cuentan eventos en un intervalo (Poisson), y otras describen tiempos continuos entre eventos (Exponencial). La elección correcta de la distribución es crítica para que el modelo sea una representación fiel de la realidad.

Por otra parte, dado que las computadoras son máquinas deterministas, no pueden generar números verdaderamente aleatorios. En cambio, producen secuencias de números *pseudoaleatorios* mediante algoritmos matemáticos. Este trabajo estudia los métodos más utilizados —el de los cuadrados medios y el congruencial lineal— y las pruebas estadísticas que permiten verificar la calidad de las secuencias generadas.

Finalmente, se presenta el método de la transformada inversa, que permite generar muestras de cualquier distribución a partir de números uniformes en $[0, 1]$, cerrando el ciclo entre la generación de números aleatorios y la simulación de fenómenos reales.

2. Distribuciones Estadísticas

Una variable aleatoria es una función que asigna un valor numérico a cada resultado posible de un experimento aleatorio. Las distribuciones estadísticas describen el comportamiento probabilístico de estas variables. A continuación se presentan las siete distribuciones estudiadas.

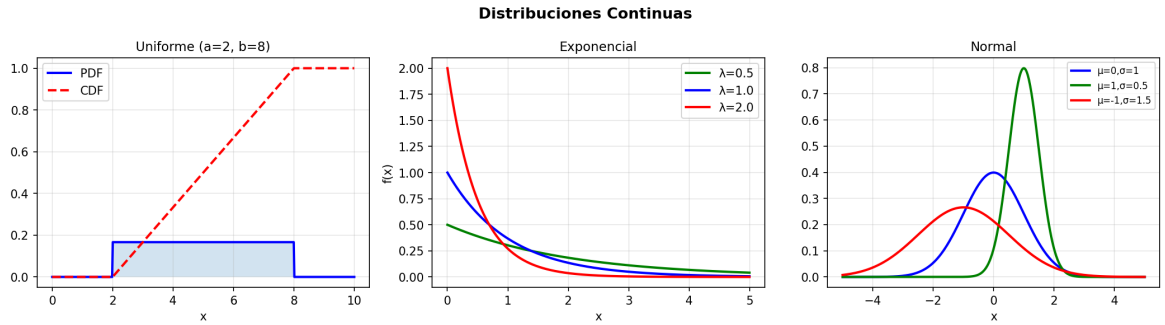

Figura 1: Distribuciones continuas: Uniforme, Exponencial y Normal (PDF en línea sólida, CDF en línea punteada).

2.1 Distribución Uniforme (continua)

Escenario de aplicación: Se utiliza cuando todos los valores dentro de un intervalo son igualmente probables. Ejemplos: simulación de tiempos de llegada sin información previa, sorteos y asignaciones aleatorias, y como base para la generación de otras distribuciones mediante la transformada inversa.

Fórmula (PDF):

$$f(x) = \frac{1}{b-a}, \quad a < x < b \qquad f(x) = 0 \text{ en otro caso}$$

Función acumulada (CDF):

$$F(x) = \begin{cases} 0 & x \leq a \\ \dfrac{x-a}{b-a} & a < x < b \\ 1 & x \geq b \end{cases}$$

Características principales:

- Tipo: **continua**
- Media: $E[X] = \dfrac{a+b}{2}$
- Varianza: $\mathrm{Var}(X) = \dfrac{(b-a)^2}{12}$
- Moda: cualquier valor en $[a, b]$ (distribución plana, sin pico)

2.2 Distribución Triangular (continua)

Escenario de aplicación: Útil en gestión de proyectos (metodología PERT) cuando se conoce el valor mínimo, máximo y más probable. También se usa en simulaciones de costos y duraciones cuando los datos son escasos y se consulta a un experto para estimar tres valores: optimista, pesimista y más probable.

Fórmula (PDF): con mínimo a, moda b, máximo c:

$$f(x) = \begin{cases} \dfrac{2(x-a)}{(c-a)(b-a)} & a \leq x < b \\ \dfrac{2(c-x)}{(c-a)(c-b)} & b \leq x \leq c \\ 0 & \text{en otro caso} \end{cases}$$

Características principales:

- Tipo: **continua**
- Media: $E[X] = \dfrac{a+b+c}{3}$
- Varianza: $\mathrm{Var}(X) = \dfrac{a^2+b^2+c^2-ab-ac-bc}{18}$
- Moda: b

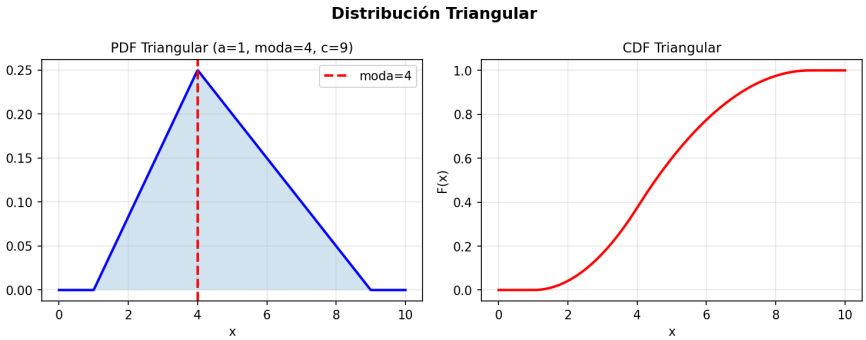

Figura 2: Distribución Triangular con $a = 1$, moda$= 4$, $c = 9$.

2.3 Distribución Exponencial (continua)

Escenario de aplicación: Modela el tiempo entre eventos que ocurren a tasa constante. Ejemplos: tiempo entre llegadas de clientes a una cola (Proceso de Poisson), tiempo de vida útil de componentes electrónicos, tiempo entre llamadas en un call center.

Fórmula (PDF):

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

Función acumulada (CDF):

$$F(x) = 1 - e^{-\lambda x}, \quad x \geq 0$$

Características principales:

- Tipo: **continua**
- Media: $E[X] = \dfrac{1}{\lambda}$
- Varianza: $\mathrm{Var}(X) = \dfrac{1}{\lambda^2}$
- Moda: 0
- Propiedad clave: **sin memoria** — $P(X > a + b \mid X > a) = P(X > b)$

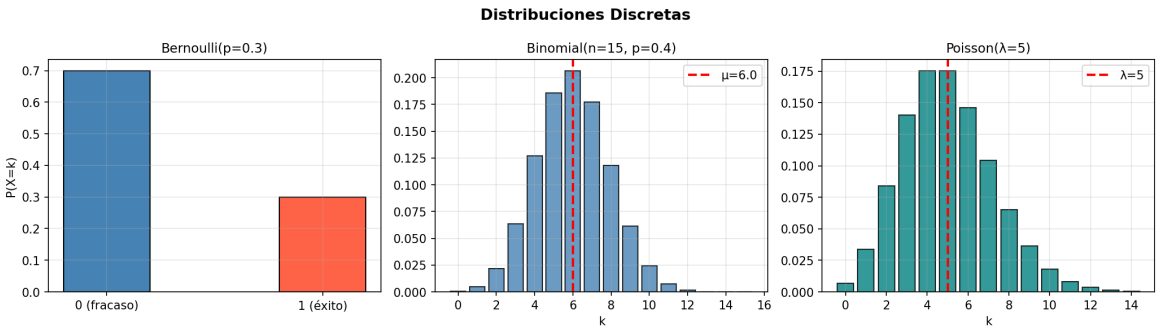

Figura 3: Distribuciones discretas: Bernoulli, Binomial y Poisson.

2.4 Distribución Bernoulli (discreta)

Escenario de aplicación: Modela un único experimento con dos posibles resultados: éxito (1) o fracaso (0). Ejemplos: lanzamiento de una moneda, test de calidad (defectuoso/no defectuoso), presencia o ausencia de una enfermedad.

Fórmula (PMF):

$$P(X = k) = p^k (1 - p)^{1-k}, \quad k \in \{0, 1\}$$

Características principales:

- Tipo: **discreta**
- Media: $E[X] = p$
- Varianza: $\mathrm{Var}(X) = p(1 - p)$
- Moda: 1 si $p > 0{,}5$; 0 si $p < 0{,}5$
- Relación: la Binomial es suma de n variables Bernoulli independientes

2.5 Distribución Binomial (discreta)

Escenario de aplicación: Cuenta el número de éxitos en n ensayos de Bernoulli independientes. Ejemplos: número de productos defectuosos en un lote, número de clientes que aceptan una oferta de n contactados.

Fórmula (PMF):

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, \ldots, n$$

Características principales:

- Tipo: **discreta**
- Media: $E[X] = n \cdot p$
- Varianza: $\mathrm{Var}(X) = n \cdot p \cdot (1-p)$
- Relación: cuando $n \to \infty$ y $p \to 0$ con $\lambda = np$ fijo, $\mathrm{Bin}(n, p) \approx \mathrm{Poisson}(\lambda)$

2.6 Distribución Poisson (discreta)

Escenario de aplicación: Cuenta el número de eventos en un intervalo de tiempo o espacio a tasa promedio constante λ. Ejemplos: llegadas a una cola por hora, llamadas a un call center, accidentes de tráfico por día.

Fórmula (PMF):

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad k = 0, 1, 2, \ldots$$

Características principales:

- Tipo: **discreta**
- Media: $E[X] = \lambda$
- Varianza: $\mathrm{Var}(X) = \lambda$ (característica única: media = varianza)
- Relación: si los tiempos entre eventos son $\mathrm{Exp}(\lambda)$, el número de eventos en tiempo t es $\mathrm{Poisson}(\lambda t)$

2.7 Distribución Normal (continua)

Escenario de aplicación: Es la distribución más importante por el Teorema Central del Límite. Ejemplos: errores de medición, alturas y pesos de una población, rendimiento académico.

Fórmula (PDF):

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Características principales:

- Tipo: **continua**, simétrica respecto a μ
- Media = Moda = μ
- Varianza: $\mathrm{Var}(X) = \sigma^2$
- Regla empírica: 68 % en $[\mu \pm \sigma]$, 95 % en $[\mu \pm 2\sigma]$, 99.7 % en $[\mu \pm 3\sigma]$
- La CDF no tiene forma cerrada; se tabula mediante la normal estándar $Z \sim N(0, 1)$

3. Métodos de Generación de Números Aleatorios

Una computadora es una máquina determinista: dado el mismo estado inicial, siempre produce el mismo resultado. Por ello, los métodos computacionales generan números *pseudoaleatorios* — secuencias que se comportan estadísticamente como aleatorias pero que son completamente reproducibles conociendo la semilla. El objetivo es producir una secuencia de números en $[0, 1]$ con propiedades ideales de **uniformidad** e **independencia**, y con una **longitud de ciclo** (período) suficientemente grande.

3.1 Método de los Cuadrados Medios (Mid-Square)

Propuesto por John von Neumann en 1946, fue uno de los primeros métodos computacionales para la generación de números pseudoaleatorios.

Algoritmo:

1. Partir de una semilla X_0 de d dígitos (típicamente $d = 4$).
2. Calcular X_0^2 y escribirlo con $2d$ dígitos (rellenando con ceros a la izquierda).
3. Extraer los d dígitos centrales $\rightarrow X_1$.
4. Repetir con X_1 para obtener $X_2, X_3, \ldots$
5. El número pseudoaleatorio en $[0, 1)$ es $U_i = X_i/10^d$.

Ejemplo con semilla $X_0 = 1234$:

$$1234^2 = 01\mathbf{5227}56 \Rightarrow X_1 = 5227 \Rightarrow U_1 = 0{,}5227$$

$$5227^2 = 27\mathbf{3215}29 \Rightarrow X_2 = 3215 \Rightarrow U_2 = 0{,}3215$$

Ventajas: conceptualmente simple y fácil de implementar.

Desventajas: es un **mal generador**. Sus propiedades estadísticas no están bien caracterizadas. El resultado depende fuertemente de la semilla: ciertas semillas producen ciclos muy cortos o colapsan rápidamente a cero (por ejemplo, la semilla 2100 colapsa de inmediato). No se recomienda para simulaciones serias.

3.2 Método Congruencial Lineal (LCG)

Es el método más ampliamente utilizado en simulación. Genera una secuencia de enteros mediante la siguiente relación recursiva:

$$y_n = (a \cdot y_{n-1} + c) \text{ mód } M$$

Los números en $[0, 1)$ se obtienen como $U_i = y_i/M$. La secuencia tiene a lo sumo M valores distintos antes de repetirse. El número K en que la secuencia comienza a repetirse se denomina **período**.

Ejemplo numérico con $a = 17$, $c = 43$, $m = 100$, $X_0 = 27$:

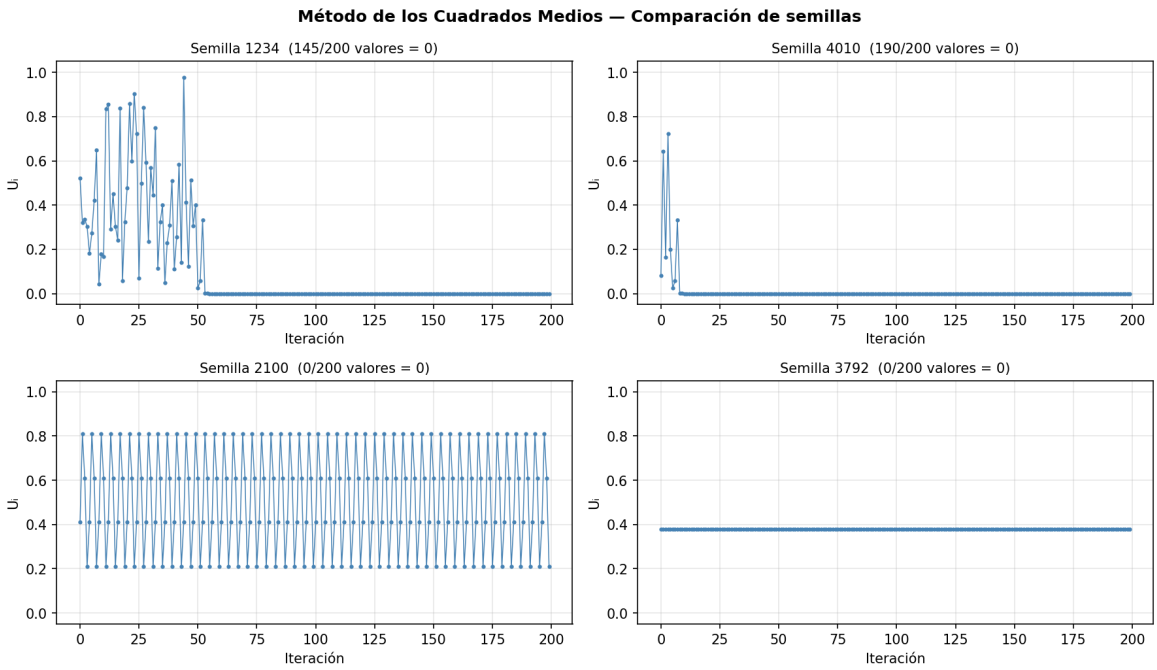

Figura 4: Comparación del método Mid-Square con distintas semillas. La semilla 2100 colapsa inmediatamente a cero.

Parámetro	Nombre	Descripción
a	Multiplicador	Controla la dispersión de la secuencia
c	Incremento	Desplazamiento aditivo
M	Módulo	Techo de la secuencia; determina el período máximo
y_0	Semilla	Valor inicial; determina toda la secuencia

Cuadro 1: Parámetros del generador congruencial lineal.

i	X_i	Cálculo	$R_i = X_i/m$
0	27	(semilla)	—
1	2	$(17 \times 27 + 43) = 502$ mód 100	0.02
2	77	$(17 \times 2 + 43) = 77$ mód 100	0.77
3	52	$(17 \times 77 + 43) = 1352$ mód 100	0.52

Cuadro 2: Ejemplo numérico del LCG con parámetros del material de cátedra.

Parámetros Lewis-Learmonth (referencia del material de clase): $M = 2^{31} - 1$, $a = 7^5 = 16807$, $c = 0$. Aseguran período $P = M - 1 \approx 2{,}1 \times 10^9$.

Ventajas: rápido, eficiente, período largo y bien estudiado, propiedades estadísticas conocidas.

Desventajas: los puntos en k dimensiones caen en hiperplanos (estructura lattice). No es criptográficamente seguro.

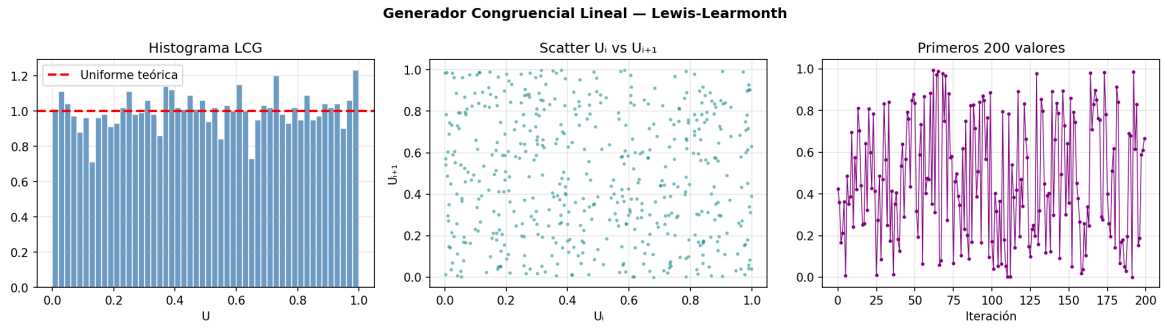

Figura 5: Generador LCG (Lewis-Learmonth): histograma uniforme, independencia visual y serie temporal.

4. Variantes del Método Congruencial

La fórmula general $y_n = (a \cdot y_{n-1} + c) \bmod M$ da lugar a tres variantes según los valores de a y c:

Variante	Condición	Fórmula	Período máximo
Multiplicativo	$c = 0$	$y_n = a \cdot y_{n-1} \bmod M$	$M - 1$
Aditivo	$a = 1$	$y_n = (y_{n-1} + c) \bmod M$	$M/\gcd(c, M)$
Mixto	$a \neq 0,\ c \neq 0$	$y_n = (a \cdot y_{n-1} + c) \bmod M$	M

Cuadro 3: Comparación de las tres variantes del método congruencial.

4.1 Congruencial Multiplicativo ($c = 0$)

Ventajas: más rápido que el mixto (una operación menos). Con la elección correcta de a, puede alcanzar período $M - 1$.

Desventajas: la semilla $y_0 = 0$ produce la secuencia $0, 0, 0, \ldots$ y debe evitarse obligatoriamente. El período máximo es $M - 1$ en lugar de M.

4.2 Congruencial Aditivo ($a = 1$)

Ventajas: implementación trivial, extremadamente rápido.

Desventajas: la secuencia es muy predecible y tiene baja calidad estadística. El período es $M/\gcd(c, M)$, que se maximiza cuando c y M son coprimos. No se recomienda para simulación seria.

4.3 Congruencial Mixto (caso general)

Ventajas: puede alcanzar período completo M bajo las condiciones del **Teorema de Hull-Dobell**:

1. $\gcd(c, M) = 1$
2. $a \equiv 1 \pmod{p}$ para todo primo p que divide a M
3. $a \equiv 1 \pmod{4}$ si 4 divide a M

Es el más versátil y recomendado.

Desventajas: levemente más lento que el multiplicativo.

4.4 Necesidad de los Tests de Verificación

Aunque los métodos congruenciales son deterministas, deben comportarse estadísticamente como si fueran aleatorios para ser útiles en simulación. Los principales problemas que pueden aparecer son:

- Los números no están uniformemente distribuidos en $[0, 1]$.
- Existe correlación entre valores consecutivos (falta de independencia).
- La media o varianza observadas difieren de los valores teóricos (0,5 y 1/12).
- El período es corto, generando variaciones cíclicas detectables.

Estos defectos **no siempre son visibles a simple vista**, por lo que los tests estadísticos son esenciales para verificar objetivamente la calidad de la secuencia antes de utilizarla en una simulación.

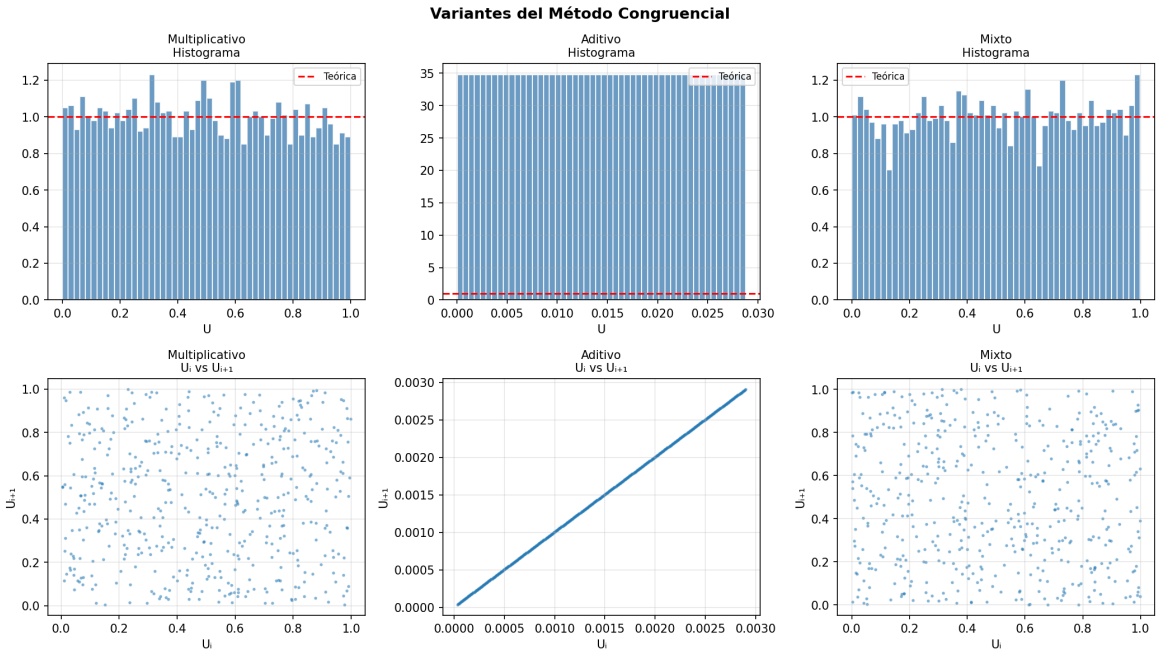

Figura 6: Comparación de las tres variantes congruenciales: histogramas y scatter U_i vs U_{i+1}.

5. Pruebas de Aleatoriedad

Para que una secuencia de números pseudoaleatorios sea útil en simulación debe verificar dos propiedades fundamentales: **uniformidad** (los valores deben estar uniformemente distribuidos en $[0, 1]$) e **independencia** (los valores deben ser estadísticamente independientes entre sí).

5.1 Prueba de Uniformidad (Chi-cuadrado)

Descripción: divide el intervalo $[0, 1]$ en k subintervalos iguales y compara las frecuencias observadas O_i con las frecuencias esperadas $E_i = N/k$ bajo uniformidad.

Estadístico:

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$$

Criterio: se rechaza la hipótesis de uniformidad si $\chi^2 > \chi^2_{\alpha,\, k-1}$.

Propiedad verificada: uniformidad de la distribución.

5.2 Prueba de Kolmogorov-Smirnov (KS)

Descripción: mide la máxima distancia entre la función de distribución empírica $F_n(x)$ y la CDF teórica $F(x) = x$:

$$D_n = \sup_x |F_n(x) - F(x)|$$

Ventaja sobre Chi-cuadrado: no requiere agrupar en intervalos; trabaja directamente con la distribución continua.

Propiedad verificada: ajuste global a la distribución uniforme.

5.3 Prueba de Independencia — Autocorrelación

Descripción: calcula la correlación entre U_i y U_{i+h} para distintos desfasajes h. Si los valores son independientes, las autocorrelaciones deben estar cerca de cero para todo h.

Criterio: los valores deben estar dentro de la banda $\pm 1{,}96/\sqrt{N}$ para $\alpha = 0{,}05$.

Propiedad verificada: independencia entre valores de la secuencia.

5.4 Prueba de la Corrida (Runs Test)

Descripción: una *corrida* es una secuencia ininterrumpida de valores consecutivos todos por encima o todos por debajo de la mediana (0.5). Se cuenta el número total de corridas R y se compara con el esperado bajo independencia:

$$Z = \frac{R - \mu_R}{\sigma_R}, \qquad \mu_R = \frac{2n_1 n_2}{n_1 + n_2} + 1$$

donde n_1 y n_2 son la cantidad de valores por encima y por debajo de 0.5.

Criterio: se rechaza la hipótesis de independencia si $|Z| > 1{,}96$ ($\alpha = 0{,}05$).

Propiedad verificada: independencia; detecta tendencias y alternancia sistemática.

5.5 Prueba de Repetitividad

Descripción: verifica que la secuencia pseudoaleatoria no entre en ciclos cortos detectables, es decir, que el **período** sea suficientemente largo. Una secuencia de período corto se repetiría rápidamente introduciendo patrones sistemáticos en la simulación.

Algoritmo:

1. Recorrer la secuencia $U_1, U_2, \ldots, U_N$ buscando la primera reaparición de U_1.
2. La posición k donde $U_k = U_1$ es el período estimado $\hat{K}$.
3. Si $\hat{K} < N$, la secuencia tiene un ciclo detectado dentro de la muestra.

Criterio: si dentro de los N valores generados se detecta una repetición exacta del valor inicial, el generador tiene un período insuficiente. Para una secuencia de calidad se debe verificar $\hat{K} \gg N$ (el ciclo no debe aparecer dentro de la muestra). Para el generador Lewis-Learmonth con $M = 2^{31} - 1 \approx 2{,}1 \times 10^9$, ninguna muestra práctica de tamaño N debería detectar la repetición.

Propiedad verificada: longitud del ciclo (período) del generador — detecta generadores con períodos patológicamente cortos.

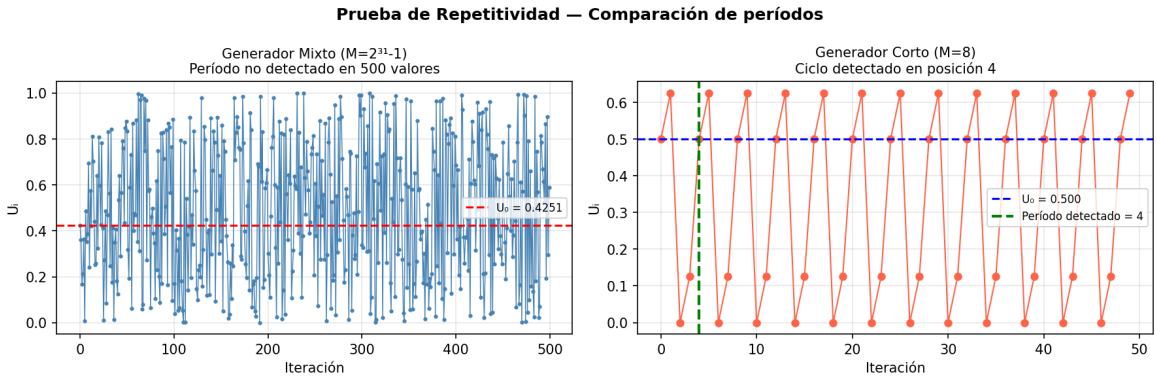

Figura 7: Prueba de Repetitividad: el generador mixto (M=$2^{31} - 1$) no repite U_0 en 500 valores; el generador con M=8 detecta el ciclo inmediatamente.

5.6 Prueba de Uniformidad de los Dígitos

Descripción: extrae el primer dígito decimal de cada U_i (es decir, $\lfloor U_i \cdot 10 \rfloor$) y verifica mediante Chi-cuadrado que cada dígito $0, 1, \ldots, 9$ aparezca con frecuencia $N/10$.

Propiedad verificada: uniformidad de los dígitos de los números pseudoaleatorios.

5.7 Prueba de Distancia

Descripción: forma puntos 2D con pares consecutivos (U_{2i}, U_{2i+1}) y analiza la distribución de las distancias mínimas entre puntos. Bajo uniformidad e independencia verdaderas, estas distancias siguen una distribución conocida.

Propiedad verificada: estructura espacial de la secuencia; detecta agrupamiento o separación anormal de puntos en el espacio bidimensional.

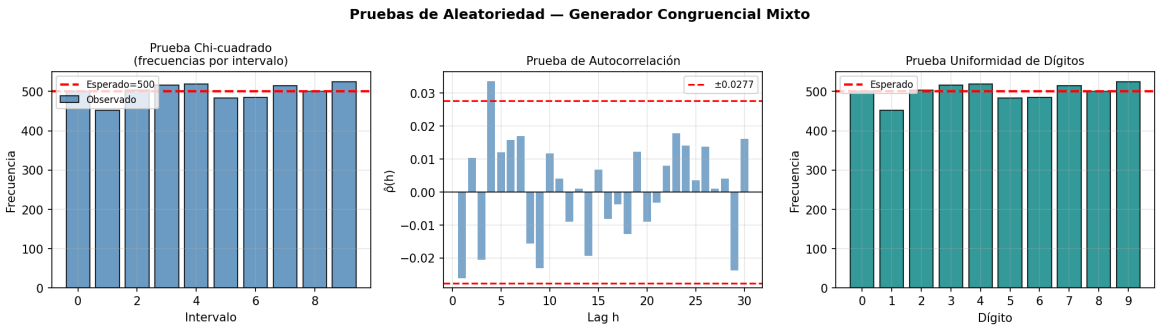

Figura 8: Resultados de tres pruebas de aleatoriedad aplicadas al generador congruencial mixto.

6. Generación de Variables Aleatorias

Una vez que disponemos de una secuencia $U_i \sim U[0,1]$ de buena calidad, el siguiente paso es generar muestras de otras distribuciones. El método fundamental para lograrlo es la **transformada inversa**.

6.1 Método de la Transformada Inversa

Fundamento teórico: si $U \sim U[0,1]$ y F es la CDF de la distribución deseada, entonces la variable $X = F^{-1}(U)$ tiene exactamente la distribución F.

Demostración:

$$P(X \leq x) = P\Big(F^{-1}(U) \leq x\Big) = P(U \leq F(x)) = F(x) \quad \checkmark$$

Algoritmo:

1. Generar $U \sim U[0,1]$ con el LCG.
2. Calcular $X = F^{-1}(U)$.

Interpretación gráfica: se grafica la CDF $F(x)$. Se genera $R \in [0,1]$, se ingresa por el eje vertical, se intercepta la curva, y el valor X correspondiente en el eje horizontal tiene la distribución deseada.

6.2 Distribución Exponencial

Con $F(x) = 1 - e^{-\lambda x}$, despejando X:

$$F(X) = 1 - e^{-\lambda X} = R \implies X = \frac{-\ln(R)}{\lambda}$$

Como $(1 - R) \sim U[0,1]$ también, se simplifica a $X = -\ln(U)/\lambda$.

6.3 Distribución Uniforme (a, b)

$$F(X) = \frac{X - a}{b - a} = R \implies X = a + (b - a) \cdot R$$

6.4 Distribución Triangular

Aplicando la transformada inversa a la CDF triangular general se obtiene (ec. 30 del material de cátedra):

$$X = \begin{cases} a + \sqrt{(b-a)(c-a)\cdot R} & 0 \le R < \dfrac{b-a}{c-a} \\ b + \sqrt{[(c-a)R - (b-a)]\,(c-b)} & \dfrac{b-a}{c-a} \le R \le 1 \end{cases}$$

El punto de corte $R_c = (b-a)/(c-a)$ separa los dos tramos de la función inversa.

6.5 Distribuciones Discretas

Para distribuciones discretas se recorre la CDF acumulada hasta encontrar el primer valor x_k tal que $F(x_k) \ge U$:

- **Bernoulli**(p)**:** $X = 1$ si $U < p$; $X = 0$ si $U \ge p$.
- **Poisson**(λ)**:** se acumula $P(X=0) + P(X=1) + \cdots$ hasta superar U, usando la recurrencia $P(X=k) = P(X=k-1)\cdot\lambda/k$.
- **Binomial**(n,p)**:** análogo al caso Poisson, recorriendo los $n+1$ valores posibles.

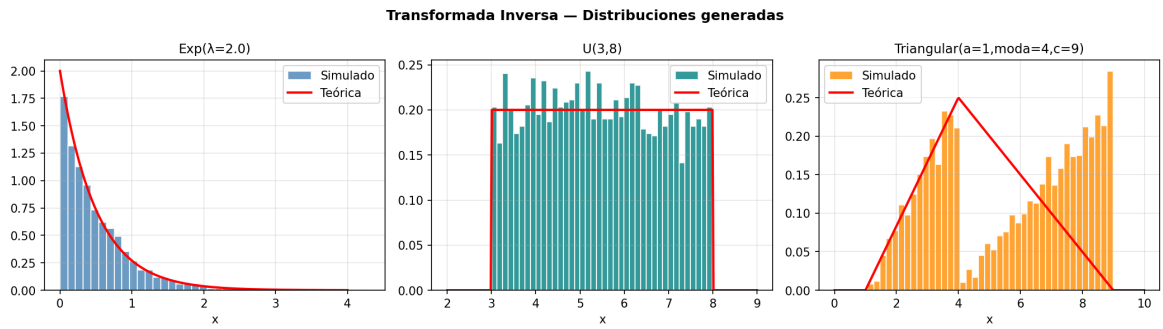

Figura 9: Generación de variables aleatorias por transformada inversa: Exponencial, Uniforme y Triangular. Las barras representan los valores simulados y la línea roja la distribución teórica.

7. Conclusiones

Este trabajo práctico recorrió los fundamentos de la simulación estocástica desde sus bases matemáticas hasta la implementación computacional en Python.

En primer lugar, se estudiaron las siete distribuciones estadísticas fundamentales. Cada una modela un tipo específico de fenómeno aleatorio: las discretas (Bernoulli, Binomial, Poisson) cuentan eventos, mientras que las continuas (Uniforme, Triangular, Exponencial, Normal) modelan magnitudes medibles. La elección correcta de la distribución es crítica para que la simulación sea representativa de la realidad.

En segundo lugar, se analizaron los métodos de generación de números pseudoaleatorios. El método de los cuadrados medios de von Neumann demostró ser un mal generador por su dependencia extrema de la semilla. El generador congruencial lineal, con los parámetros de Lewis-Learmonth, produce secuencias de alta calidad con período del orden de 2^{31}.

En tercer lugar, se implementaron las tres variantes del método congruencial y se justificó la necesidad de los tests de aleatoriedad: una secuencia puede parecer aleatoria visualmente pero presentar correlaciones o distribución no uniforme que la invaliden para simulación.

En cuarto lugar, se implementaron seis pruebas de aleatoriedad que verifican uniformidad e independencia desde distintos ángulos: Chi-cuadrado, Kolmogorov-Smirnov, autocorrelación, corridas, uniformidad de dígitos y distancia.

Finalmente, el método de la transformada inversa demostró ser el puente entre la generación de números uniformes y la simulación de cualquier distribución. Este método, junto con el generador LCG, constituye la base del futuro TP2 que vamos a realizar mas adelante, donde se utilizarán para simular un sistema de colas con un servidor.

Referencias

[1] Sheldon M. Ross (2023). *Simulation*, 6th Edition. Academic Press.

[2] Banks, J. et al. (1996). *Discrete-Event System Simulation*. Prentice Hall.

[3] Ciaburro, G. (2022). *Hands-On Simulation Modeling with Python*, 2nd Edition. Packt Publishing.

[4] Mezio, A. y Herrera, L. (2026). *Simulaciones y Procesos Estocásticos — Generación de Números Aleatorios*. Material de cátedra, UCA Rosario.

[5] Downey, A. (2023). *Modeling and Simulation in Python*. https://allendowney.github.io/ModSimPy/